

Advanced Agent Concepts

Satya Pattnaik

Agent

Multigagent orchestration

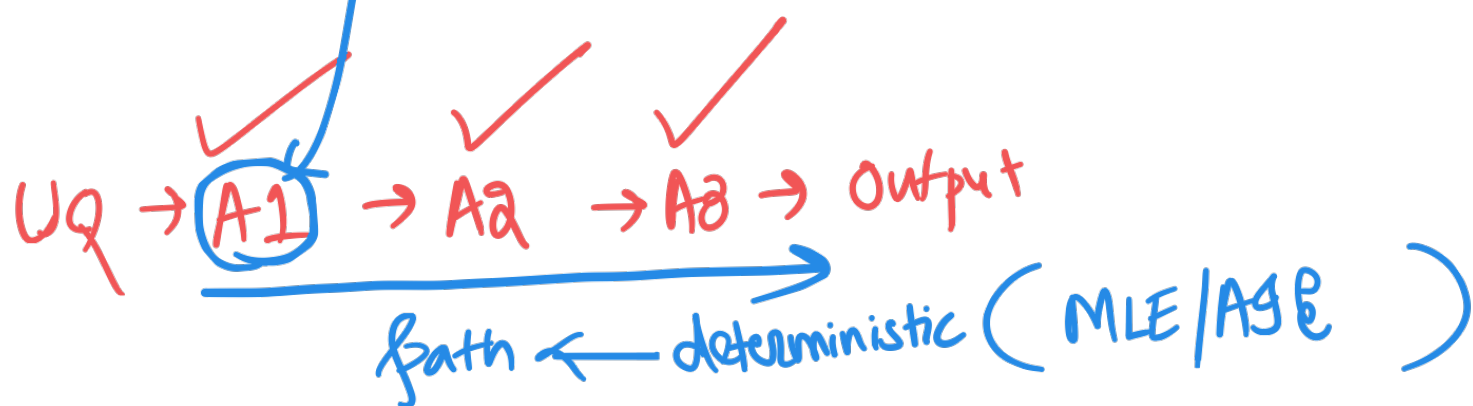
Workflow / Orchestration

Result

UQ

A₁
A₂
:
A₁₅

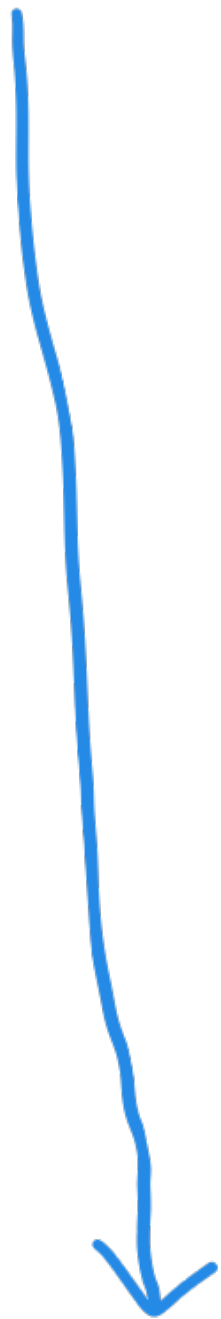
nature of this agent - stochastic

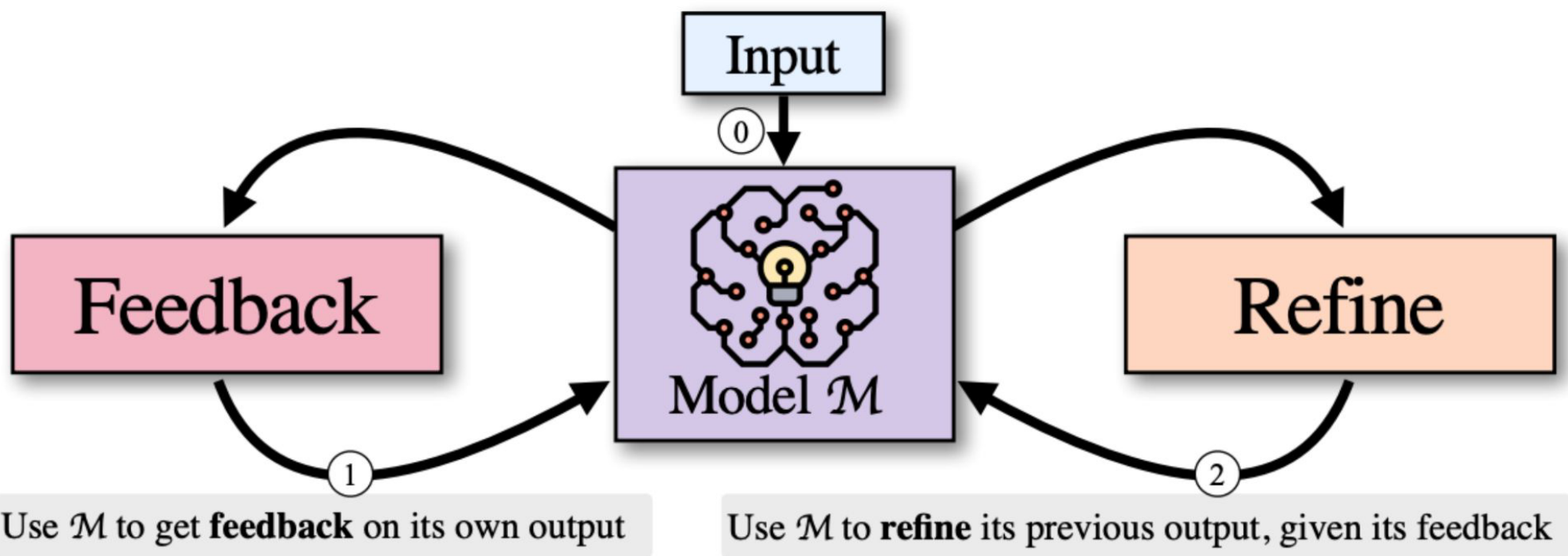


- 1] list table
- 2] table1, table2
- 3] get schema (table1)
- 4] get schema (table2)
- 5]

```
SELECT T1.COL1, T2.COL1  
FROM T1 JOIN T2  
WHERE ...
```
- 6]

```
execute - sql (
```





(a) Dialogue: x, y_t

User: I am interested in playing Table tennis. **X**

Response: I'm sure **Yes** it's a great way to socialize, stay active

(b) FEEDBACK fb

LLM feedback

Engaging: Provides no information about table tennis or how to play it.

User understanding: Lacks understanding of user's needs and state of mind.

(c) REFINE y_{t+1}

Response (**refined**): That's great to hear (...) ! It's a fun sport requiring quick reflexes and good hand-eye coordination. Have you played before, or are you looking to learn?

Yes

(d) Code optimization: x, y_t

```
Generate sum of 1, ..., N
def sum(n):
    res = 0
    for i in range(n+1):
        res += i
    return res
```

(e) FEEDBACK fb

This code is slow as it uses brute force. A better approach is to use the formula ... $(n(n+1))/2$.

(f) REFINE y_{t+1}

Code (refined)

```
def sum_faster(n):
    return (n*(n+1))//2
```

Step 1] LLM1 (Interviewer) → Question

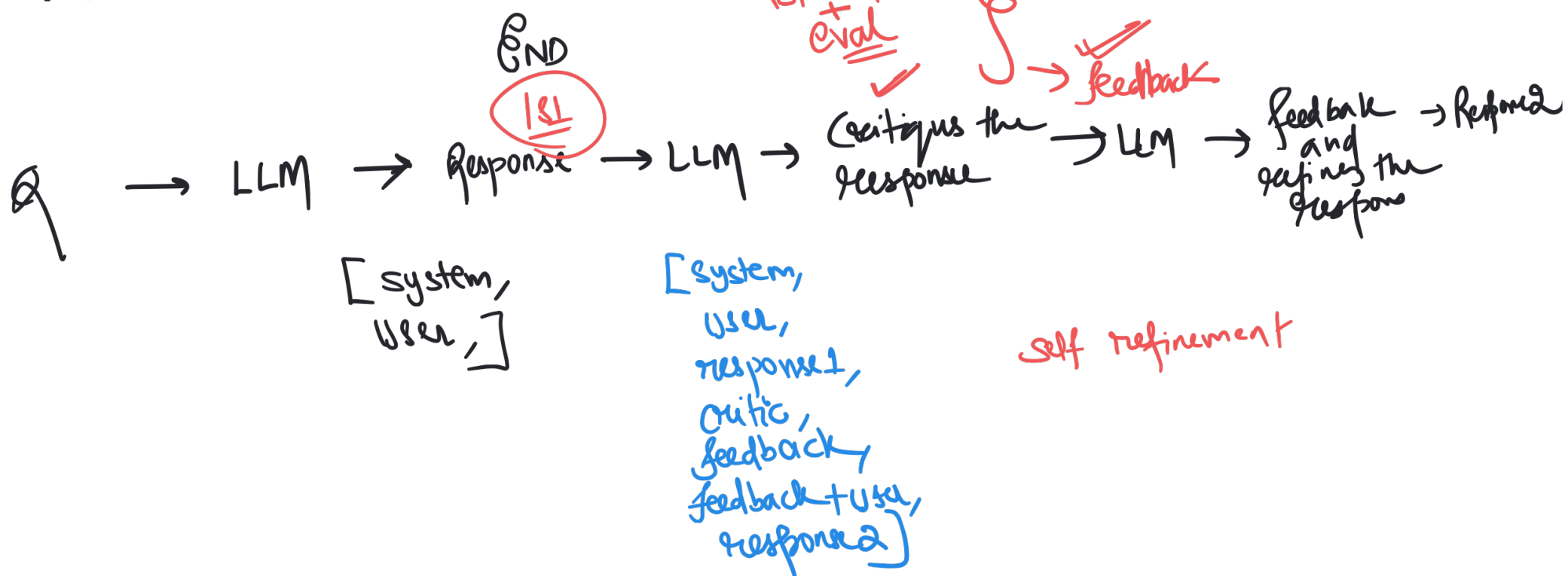
Step 2] LLM2 (Candidate) → Code

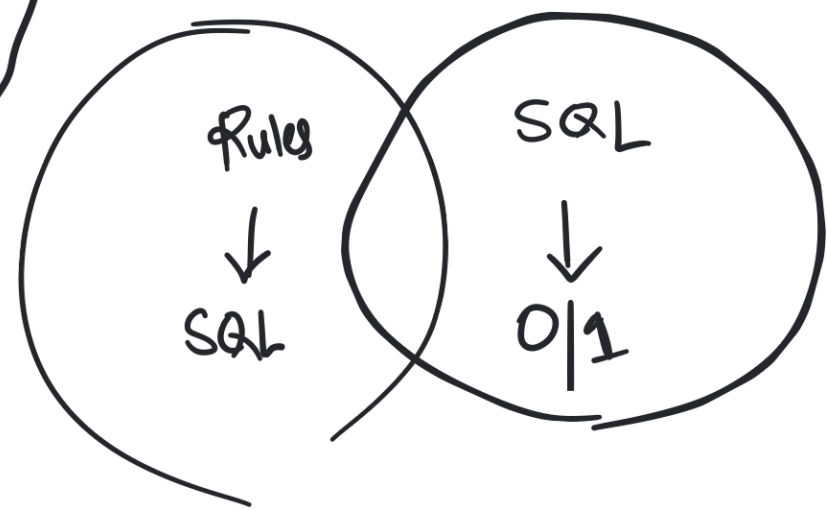
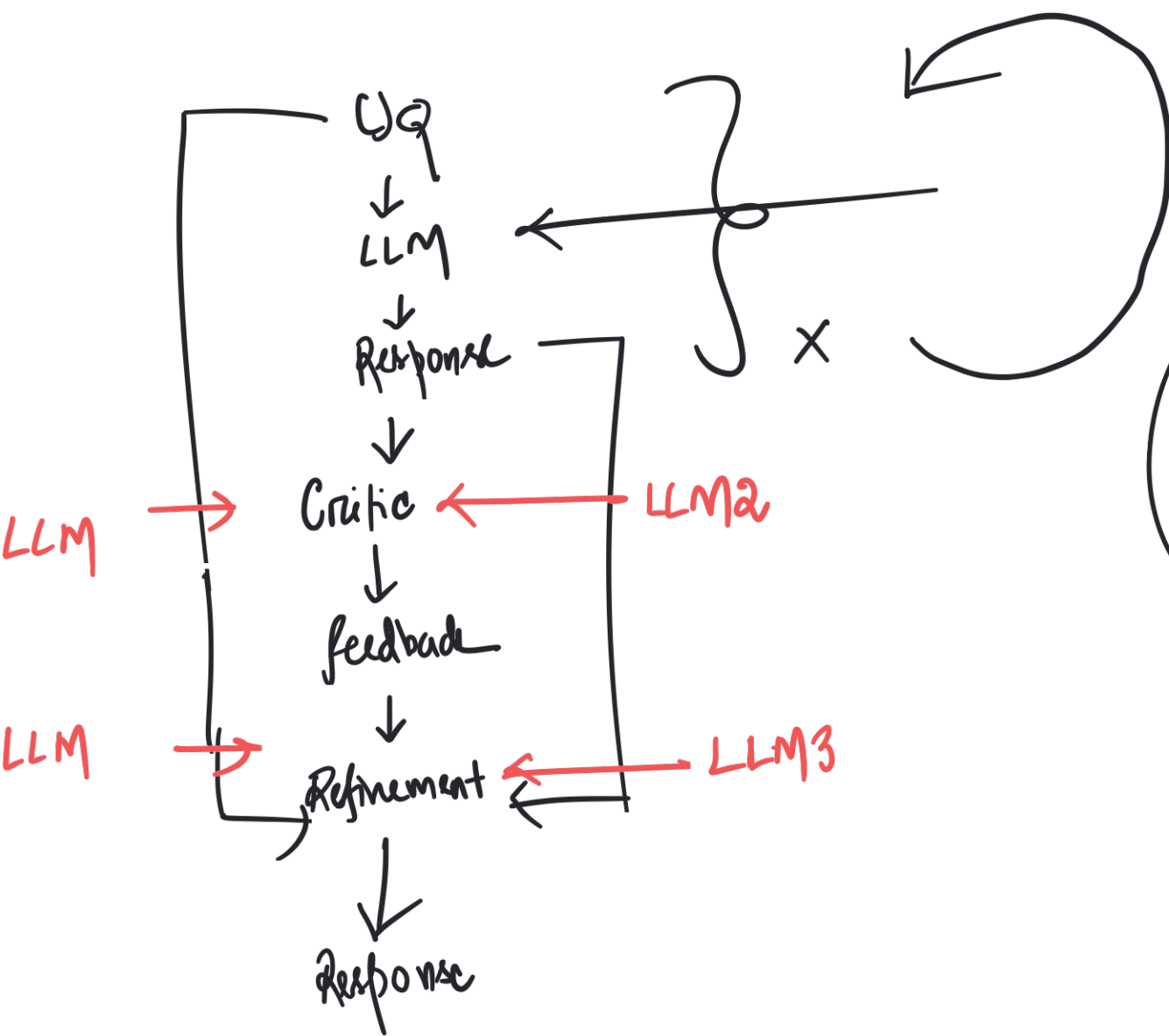
Step 3] LLM1 (Interviewer) → $O(n^2)$ — $O(n \log n)$

Step 4] LLM2 (Candidate) → Refined Code

max-iteration = 10/15

✓ eval criterion

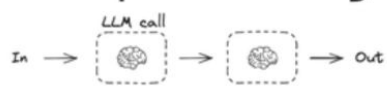




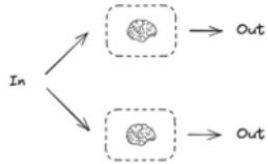
✓ Workflows (Deterministic Path)

1

Prompt Chaining

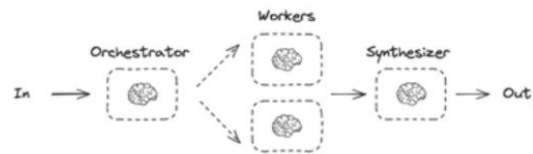


Parallelization

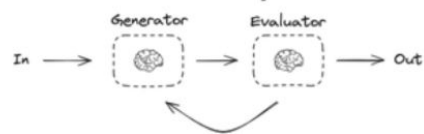


LLM is embedded in predefined code paths

Orchestrator-Worker

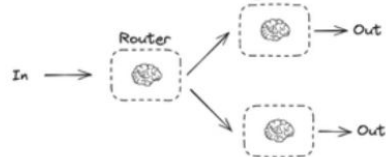


Evaluator-optimizer



2

Routing



LLM directs control flow through predefined code paths

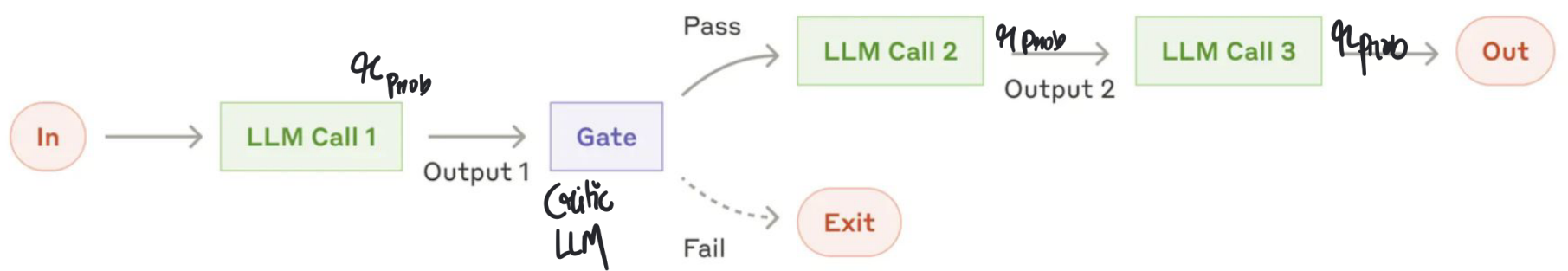
Agent (Non-determinism)



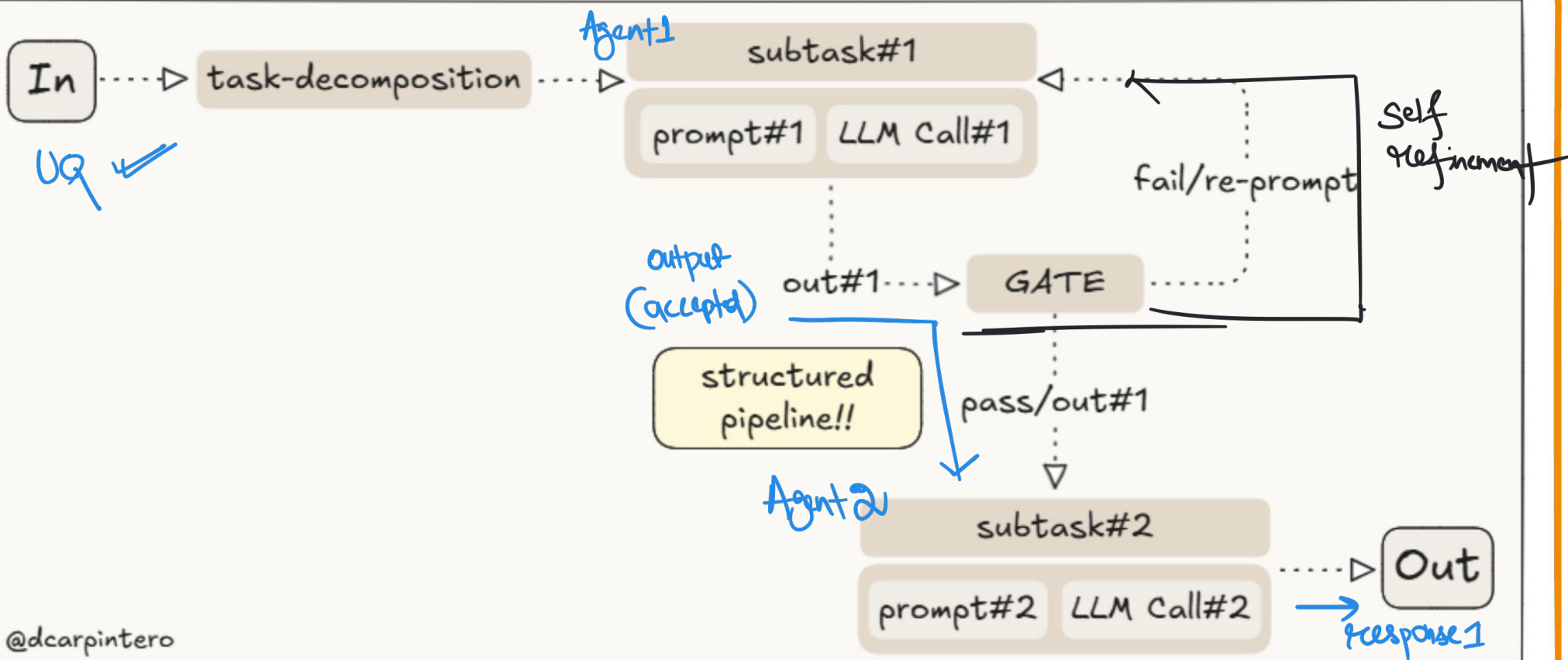
LLM directs its own actions based on environmental feedback

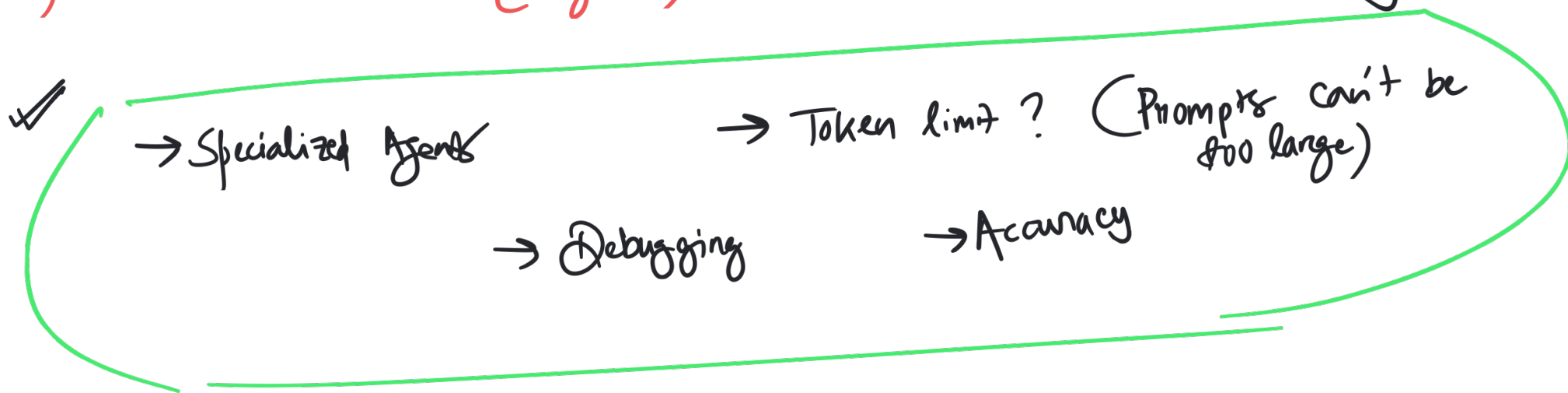
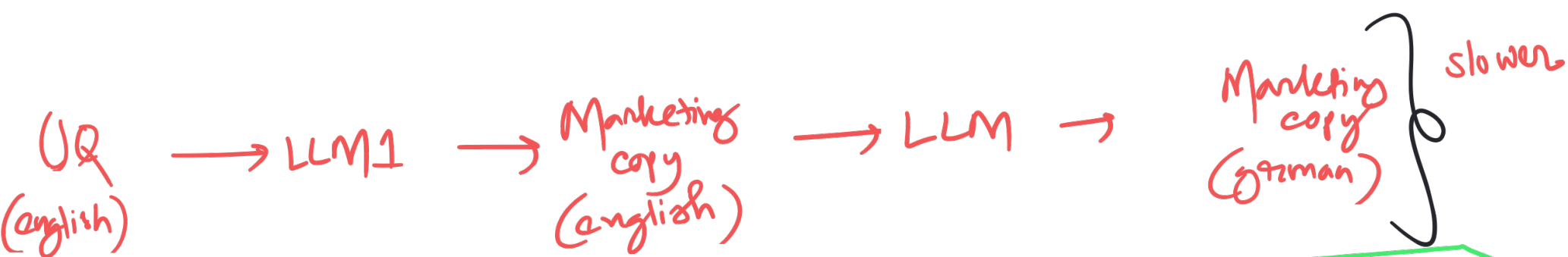
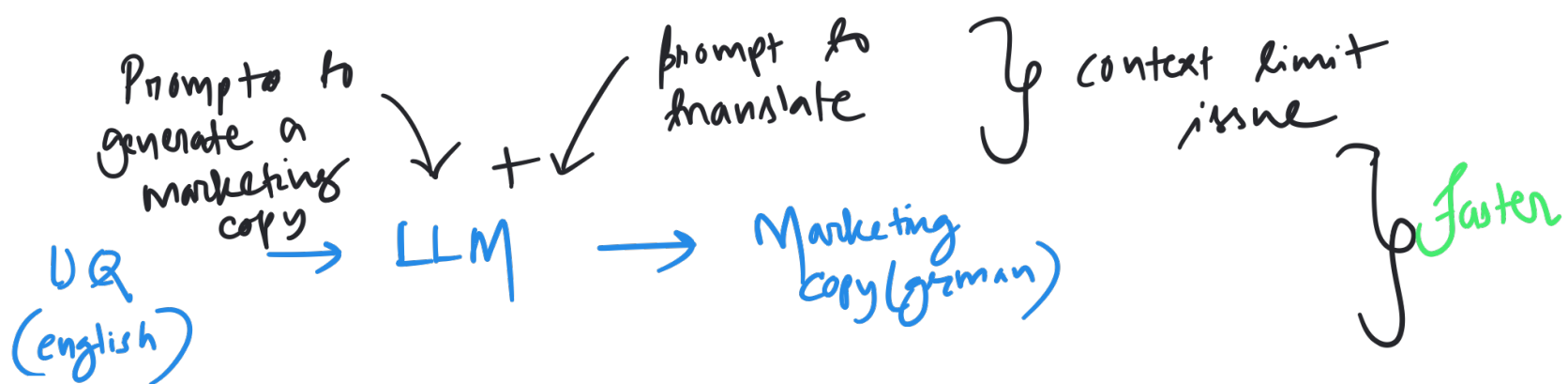
Workflow #1 → Prompt Chain

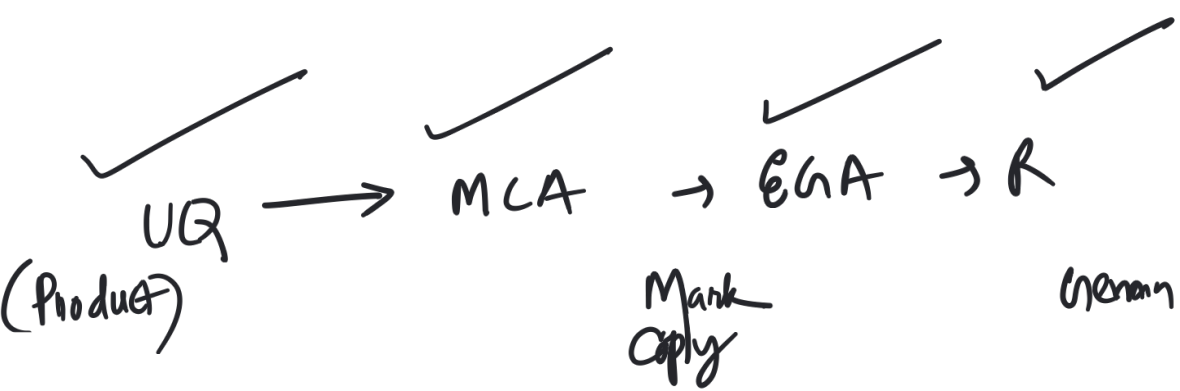
deterministic flow by developer



9:28



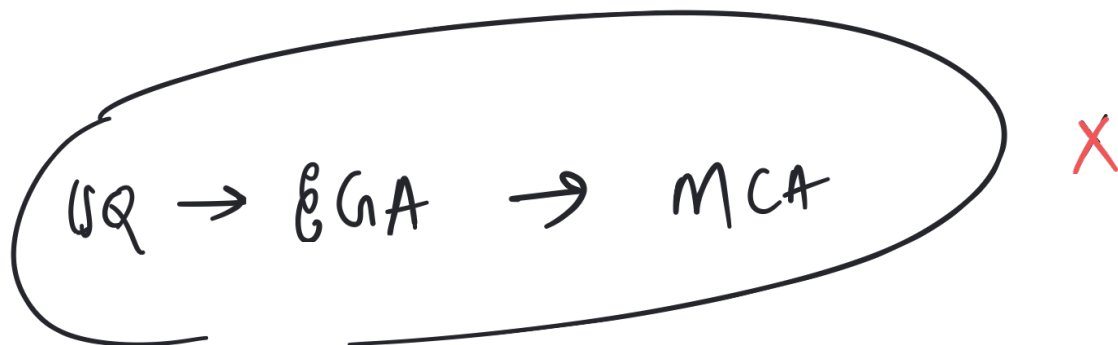




- 1] Marketing Copywriter Agent
(MCA)
- 2] Translatⁿ Agent
E $\rightarrow$ G (EGA)

LM { MCA
EGA

99%
1%.

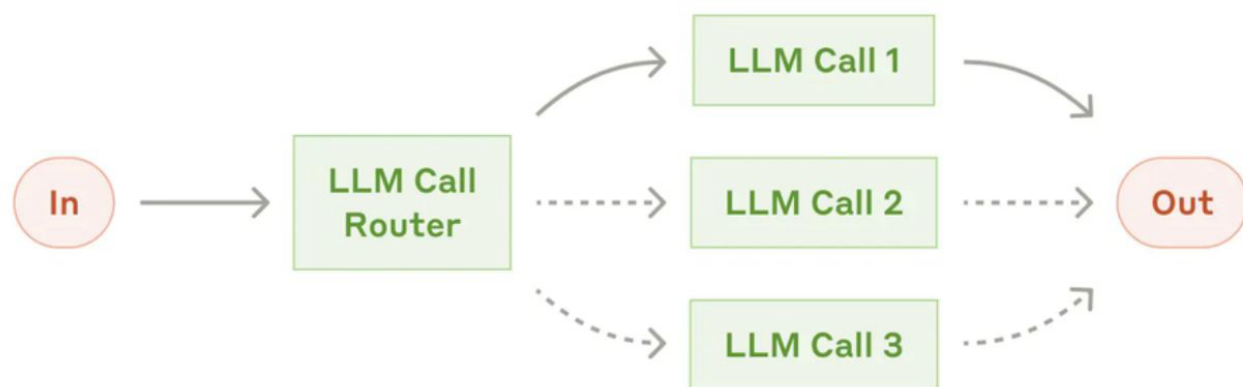


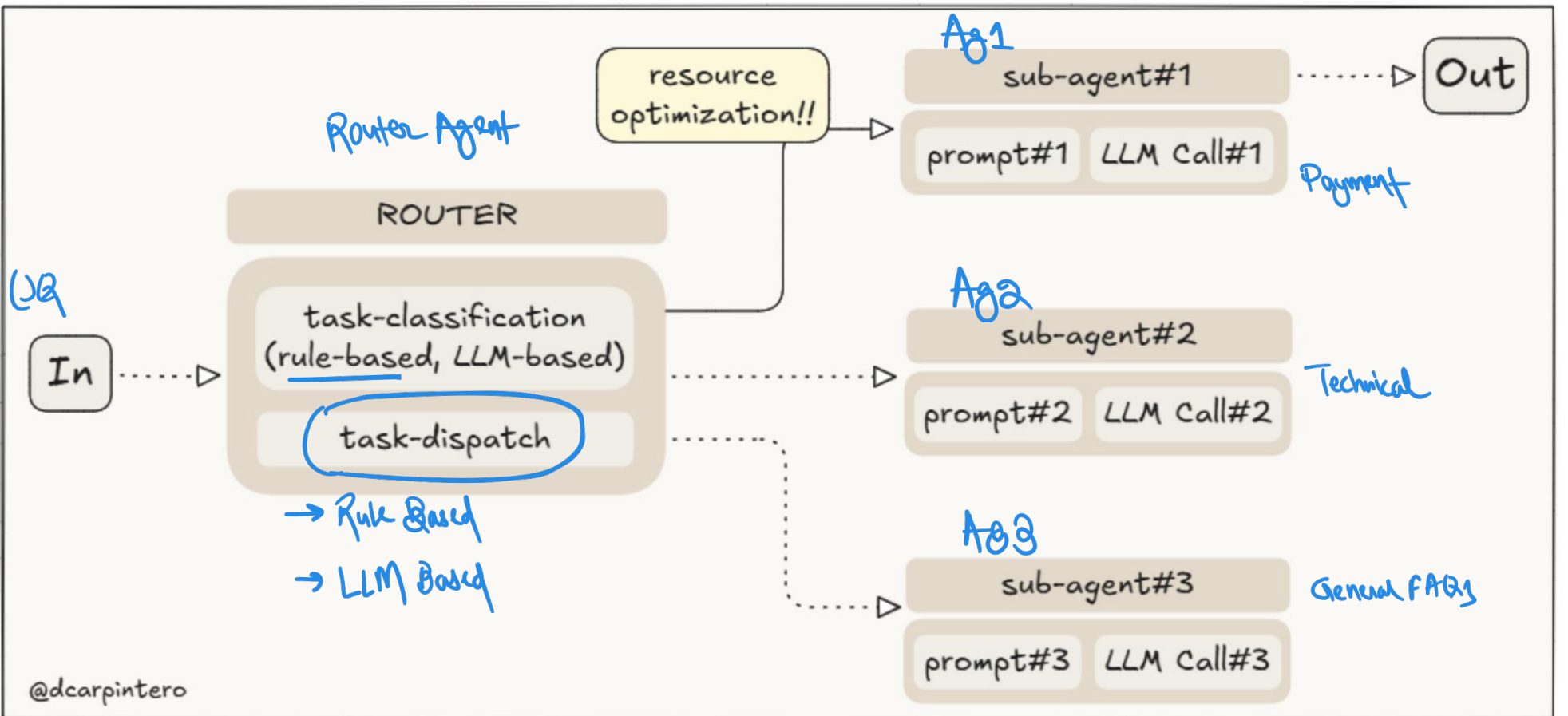
When to use this workflow: This workflow is ideal for situations where the task can be easily and cleanly decomposed into fixed subtasks. The main goal is to trade off latency for higher accuracy, by making each LLM call an easier task.

Examples where prompt chaining is useful:

- Generating Marketing copy, then translating it into a different language.
- Writing an outline of a document, checking that the outline meets certain criteria, then writing the document based on the outline.

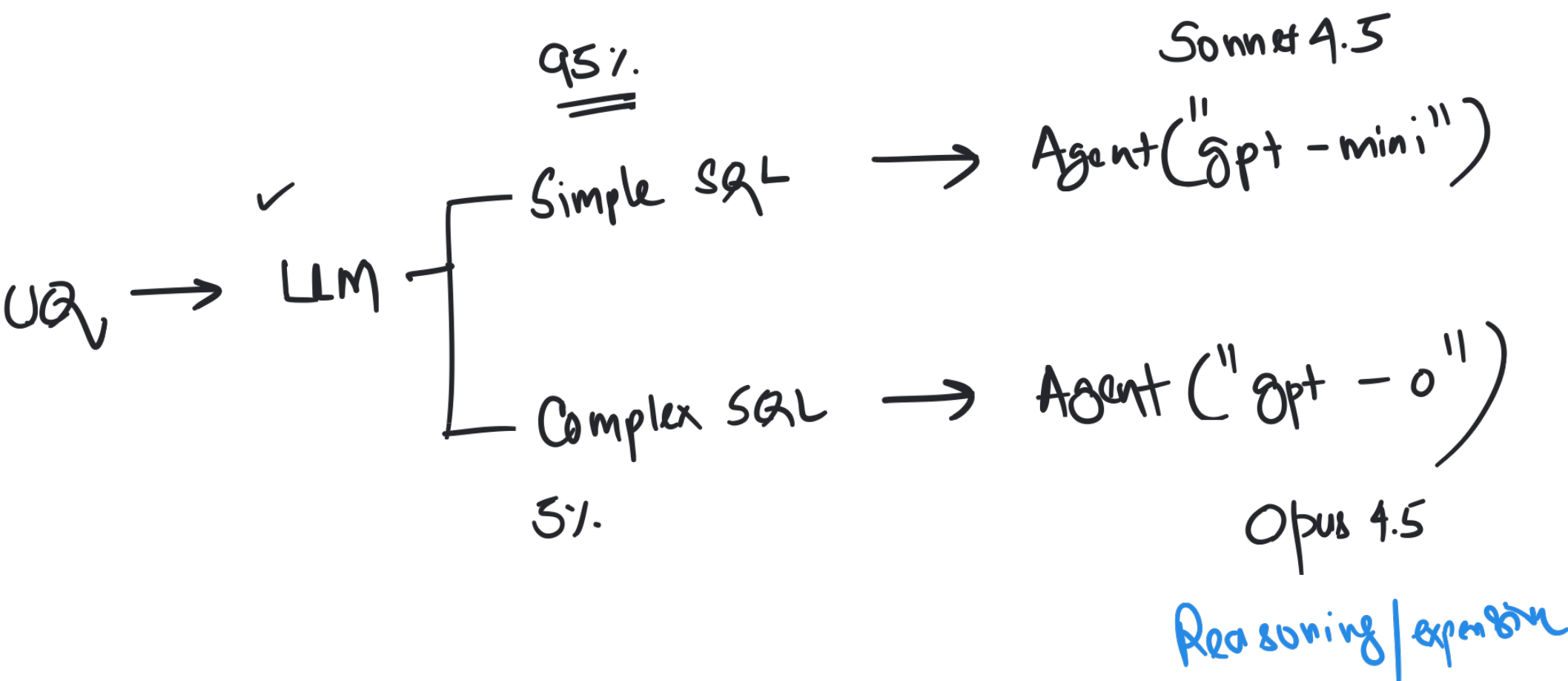
Workflow: Routing







Cheaper



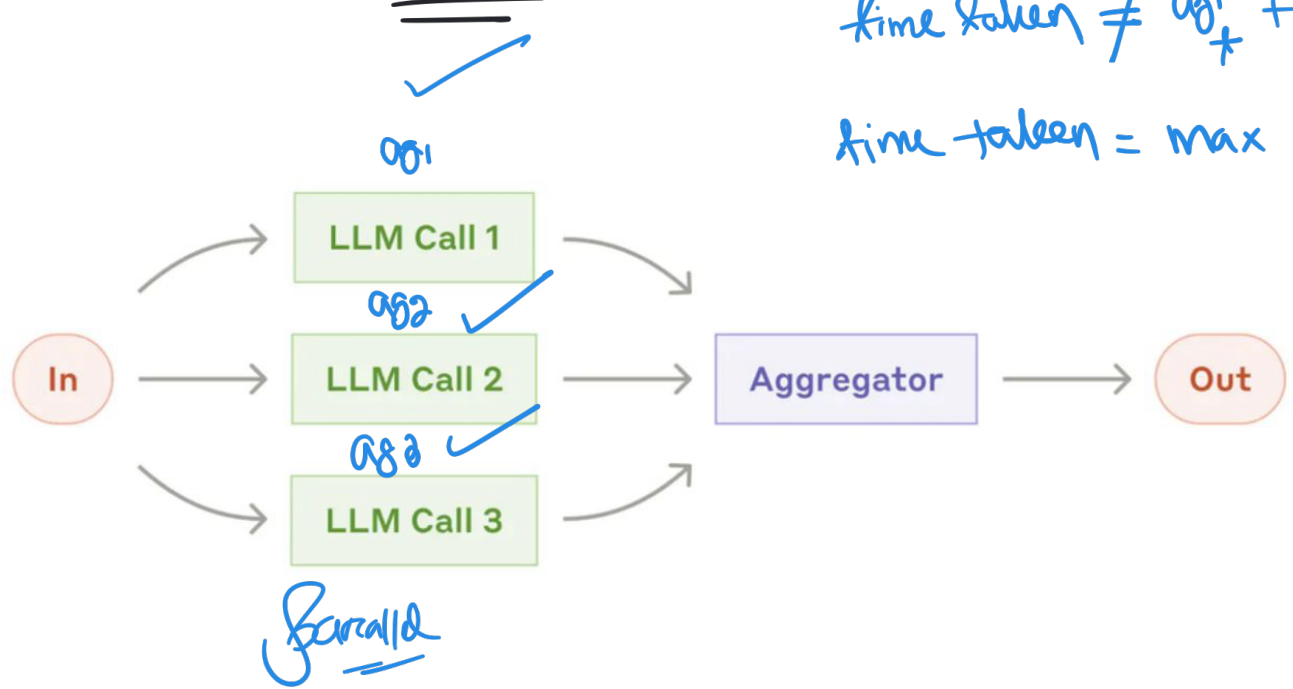
Use Cases of Routing

When to use this workflow: Routing works well for complex tasks where there are distinct categories that are better handled separately, and where classification can be handled accurately, either by an LLM or a more traditional classification model/algorithm.

Examples where routing is useful:

- Directing different types of customer service queries (general questions, refund requests, technical support) into different downstream processes, prompts, and tools.
- Routing easy/common questions to smaller, cost-efficient models like Claude Haiku 4.5 and hard/unusual questions to more capable models like Claude Sonnet 4.5 to optimize for best performance.

Parallelization



time taken $\neq ag_1^1 + ag_2^2 + ag_3^3$
time taken = $\max \begin{pmatrix} ag_1 \text{ time,} \\ ag_2 \text{ time,} \\ ag_3 \text{ time} \end{pmatrix}$

parallel

ag1
worker#1

prompt#1 LLM Call#1

ag2
worker#2

prompt#2 LLM Call#2

ag3
worker#3

prompt#3 LLM Call#3

independent
subtasks!!

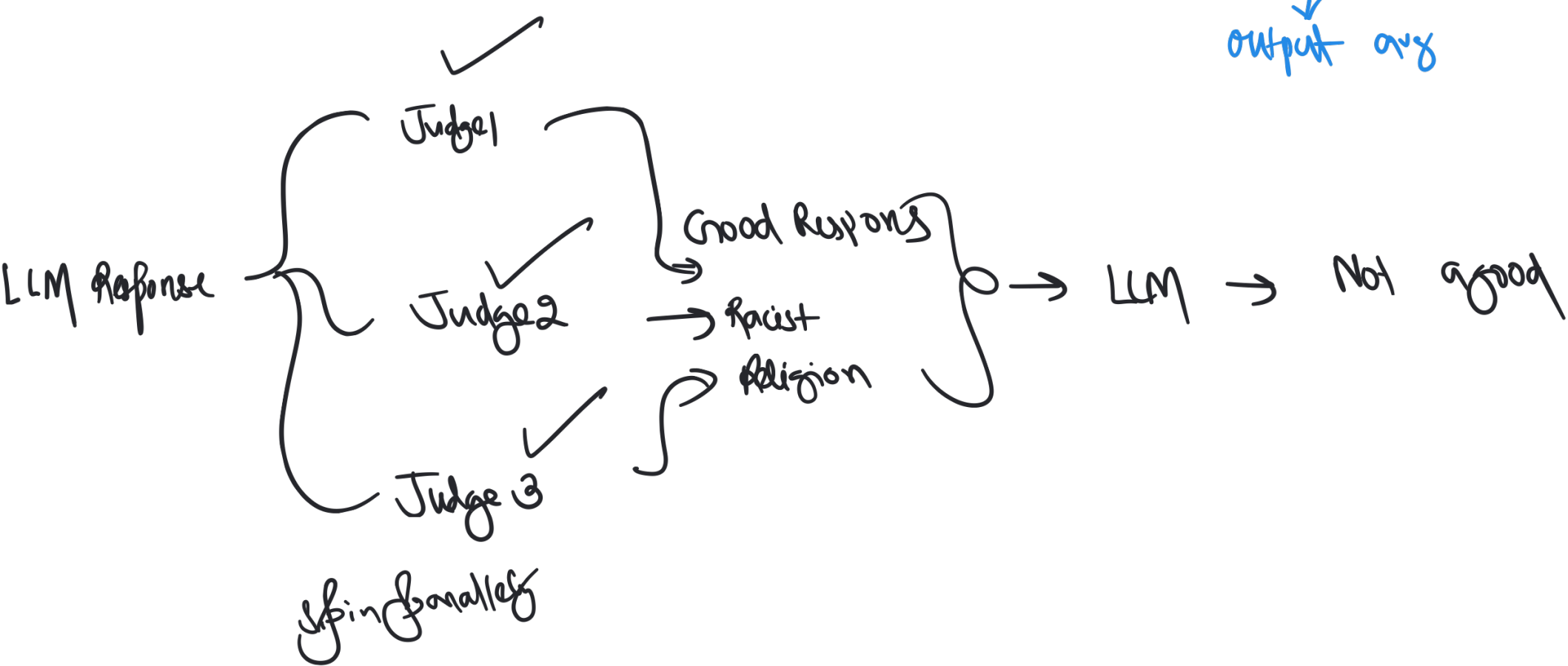
LLM
task-decomposition
sharding
aspect-based
task repetition

aggregation
concatenation
synthetizer
consensus voting

QA
In

Out

@dcarpintero



Random Forest
100 trees parallelly
↓
output avg

When to use this workflow: Parallelization is effective when the divided subtasks can be parallelized for speed, or when multiple perspectives or attempts are needed for higher confidence results. For complex tasks with multiple considerations, LLMs generally perform better when each consideration is handled by a separate LLM call, allowing focused attention on each specific aspect.

Examples where parallelization is useful:

✓ Sectioning:

- Implementing guardrails where one model instance processes user queries while another screens them for inappropriate content or requests. This tends to perform better than having the same LLM call handle both guardrails and the core response.
- Automating evals for evaluating LLM performance, where each LLM call evaluates a different aspect of the model's performance on a given prompt.

✓ • Voting:

- Reviewing a piece of code for vulnerabilities, where several different prompts review and flag the code if they find a problem.
- Evaluating whether a given piece of content is inappropriate, with multiple prompts evaluating different aspects or requiring different vote thresholds to balance false positives and negatives.

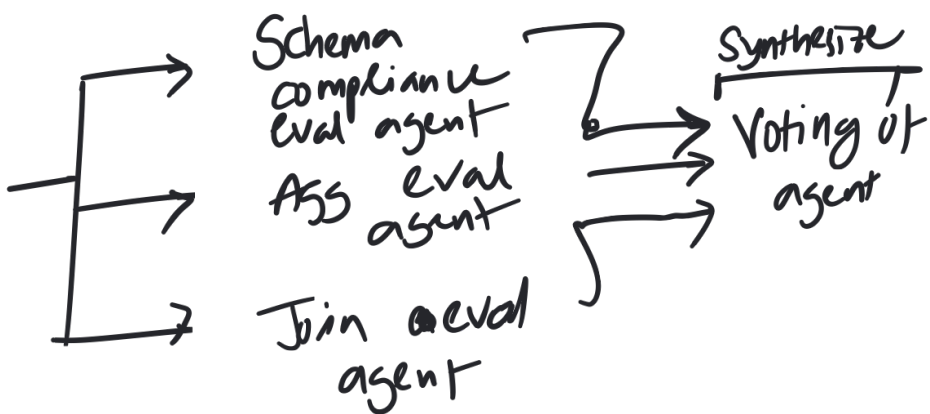
Diff agents — planning — diff tasks parallelly

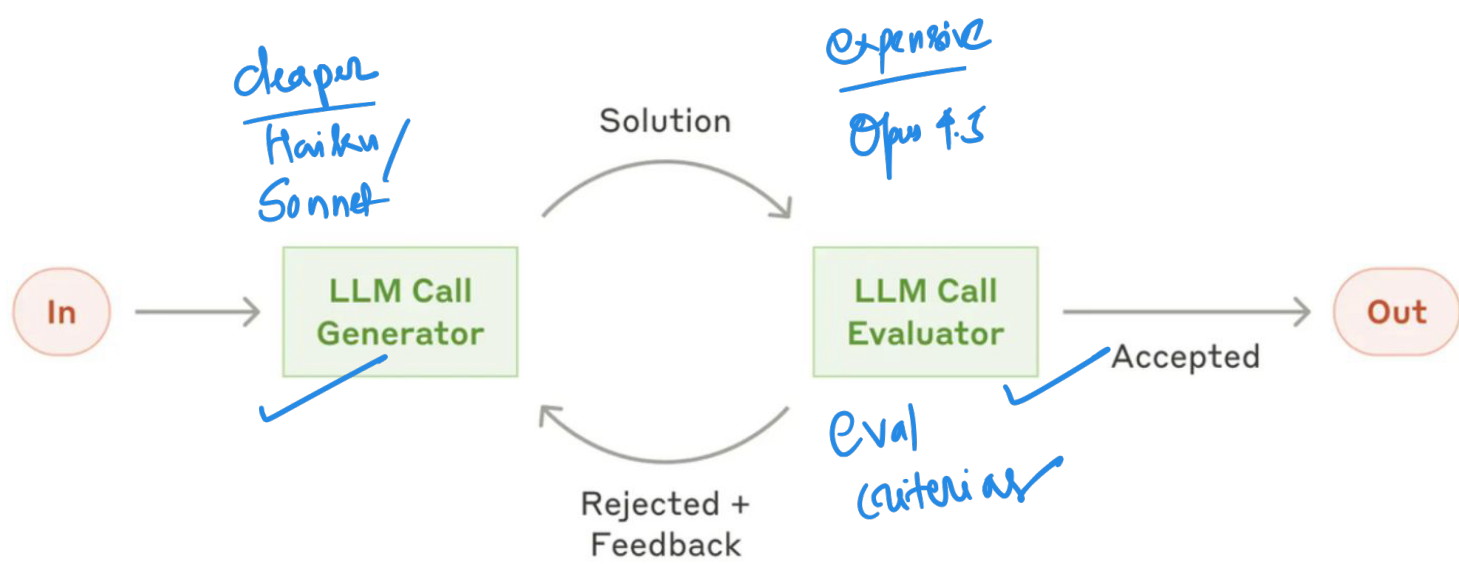
Diff agents — running ^{similar} tasks parallelly } voting

Prompt Chaining Workflow

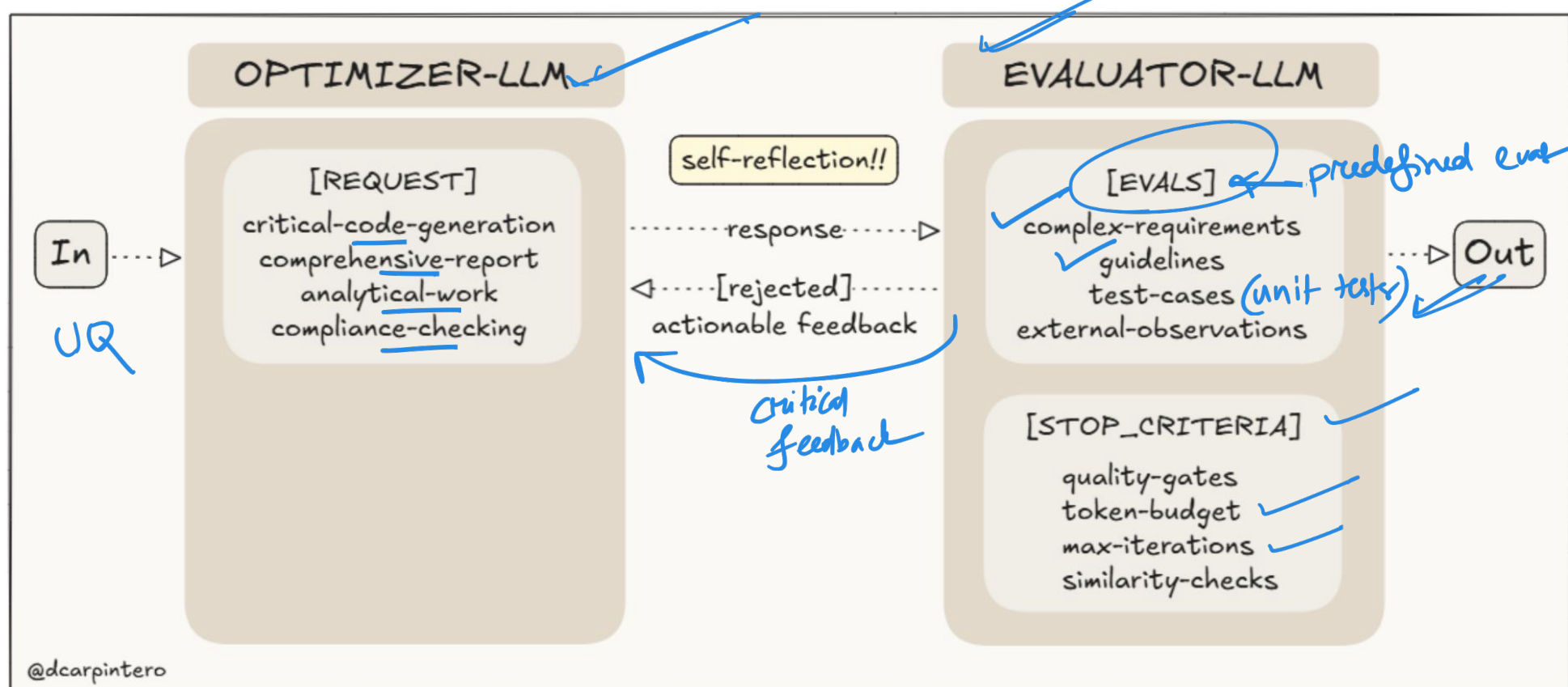
UQ → Intent Agent → Schema Link agent → SQL Gen Agent → SQL

Parallelization Workflow





loop → 10 time (stop criteria)



Guideline

When to use this workflow: This workflow is particularly effective when we have clear evaluation criteria, and when iterative refinement provides measurable value. The two signs of good fit are, first, that LLM responses can be demonstrably improved when a human articulates their feedback; and second, that the LLM can provide such feedback. This is analogous to the iterative writing process a human writer might go through when producing a polished document.

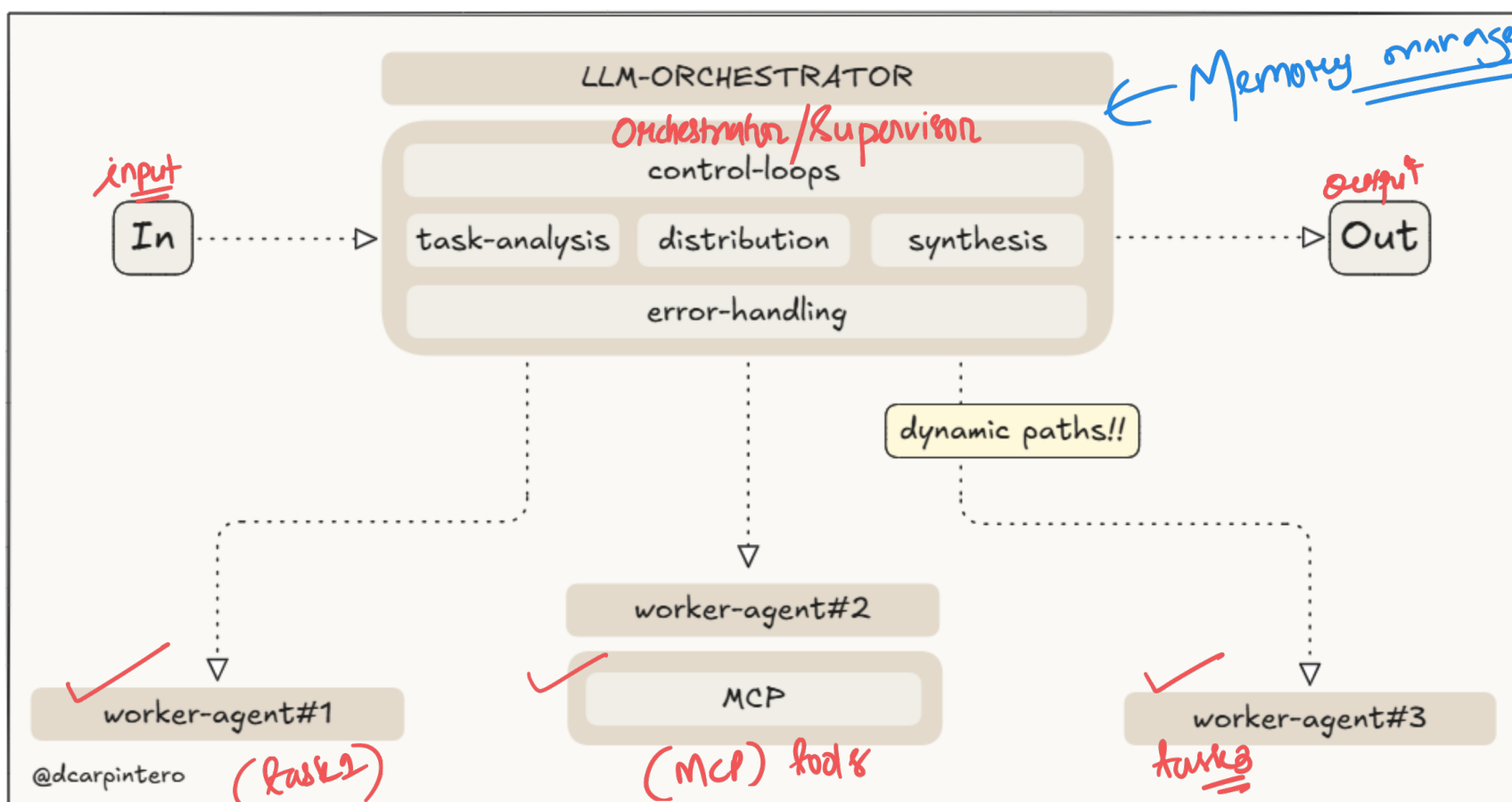
Examples where evaluator-optimizer is useful:

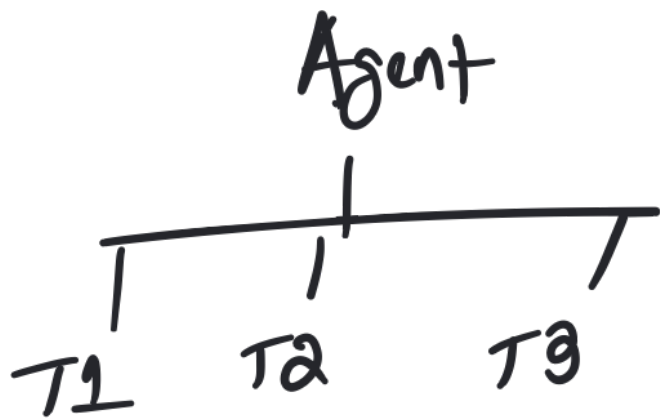
- Literary translation where there are nuances that the translator LLM might not capture initially, but where an evaluator LLM can provide useful critiques.
- Complex search tasks that require multiple rounds of searching and analysis to gather comprehensive information, where the evaluator decides whether further searches are warranted.

Code
(unit tests)
(integration
test)

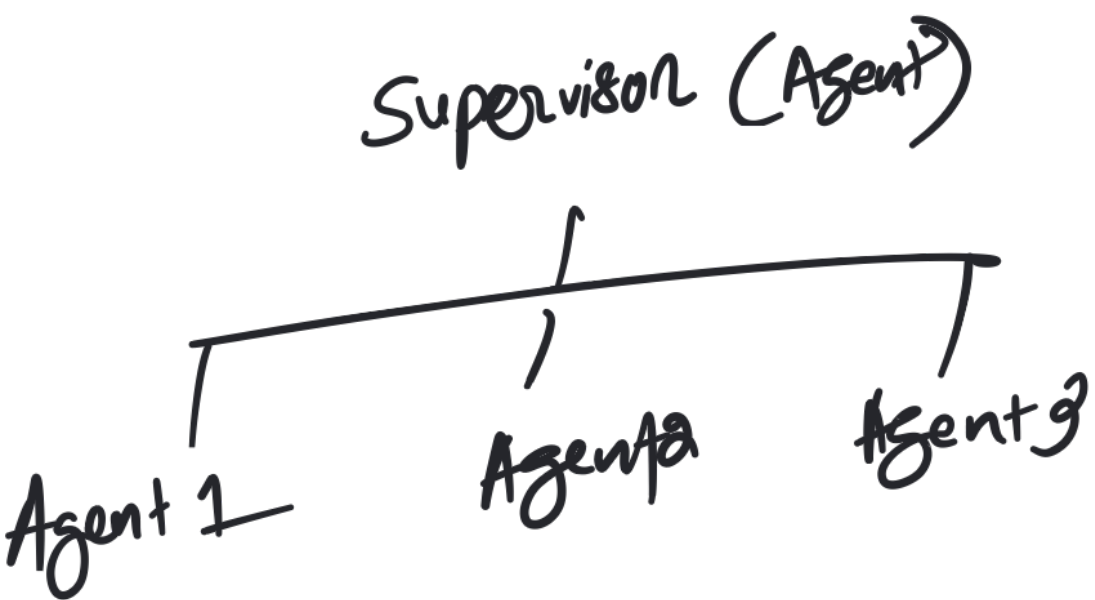
Code
($O(n^2)$)
($O(n \log n)$)
($O(n \log n)$)







AS2 Amazon
 SDE1 LC ✓✓



A1 → A2 → A3
A3 → A1 → A2

- 1] UQ
- 2] Plan
- 3] Agent to read a file (Read Agent)
- 4] ~~Agent~~ ... Processing (Processing Agent)
on the file
- 5] Write the file (Writing agent)

- 1] UQ
- 2] Plan
- DB Agent { 3] Connect to Database
- 4] Read from Database
- 5] Processing (Processing agent)
- 6] Write the file

Supervisor → AI → KB → Supervisor → of

Supervisor $\rightarrow$ A3 $\rightarrow$ A1 $\rightarrow$ Supervisor $\rightarrow$ OK

Synthesis

